

DEFENSA TÉCNICA INDIVIDUAL - PROYECTO DE VENTA DE DONUTS

SABORES FELICES DONUTIN

Nombre: Danilo Ramírez – Benjamín González

Asignatura: Desarrollo Fullstack I

Sección 004D

PARTE I – Danilo Ramírez

I. INTRODUCCIÓN

El presente documento detalla mi contribución individual en el desarrollo del proyecto del sistema de venta de donuts **Sabores Felices Donutin**. Mi rol en el proyecto abarcó desde la fase de diseño conceptual hasta la implementación de la lógica de negocio en el backend bajo una arquitectura de microservicios desacoplados, finalizando con la fase de pruebas y validación de las API REST desarrolladas.

II. DISEÑO DE DATOS

Fui el responsable de diseñar y estructurar la persistencia de datos para los módulos clave del negocio. Utilizando Oracle DataModeler creé el modelo entidad-relación y su posterior transformación al modelo relacional, asegurando la integridad y la normalización de la base de datos.

- **Estructuración de entidades:** Diseñé las entidades base con sus respectivos atributos -la mayoría obligatorios- y claves primarias (PK), para los módulos de catálogo, clientes y pedidos, este último la base del negocio.
- **Relaciones:** Implementé las siguientes relaciones entre las entidades mediante claves primarias (PK) y claves foráneas (FK):
 - a. **Categoría y donut:** **Categoría** clasifica los donuts. **Donut** guarda el detalle de cada donut, mediante la relación **Uno a muchos** (1:N). Una categoría puede tener muchos donuts, pero un donut está asociada a una categoría.
 - b. **Cliente:** Guarda los datos personales del cliente.
 - c. **Pedido y Detalle_Pedido:** Es la base del negocio. **Pedido** registra la compra realizada por el **Cliente**, mediante una relación **Uno a Muchos** (1:N). **Detalle_Pedido** registra la cantidad exacta de cada donut pedido, heredando además la clave primaria de **Pedido**.
 - d. **Pago y Comprobante_Pago:** **Pago** registra la transacción de pago del **Pedido**, mediante una relación de **Uno a Uno** (1:1). **Comprobante_Pago** modela el recibo legal emitido tras la compra, generando una relación **Uno a Uno** (1:1) con la tabla **Pago**.
 - e. **Despacho:** Se encarga de la logística del producto tras la compra, teniendo una relación **Uno a Uno** (1:1) con el **Pedido**, importando los datos del **Pedido** y del **Cliente**.

III. DESARROLLO BACKEND Y LÓGICA DE NEGOCIO EN SPRING BOOT

En la etapa de codificación, con mi colaborador nos dividimos los microservicios y desarrollé los microservicios productores de información (**Catálogo** y **Clientes**) y el receptor de la información (**Pedido**), mediante el framework **Spring Boot** aplicando el patrón arquitectónico por capas Controlador-Repositorio-Servicio para separar responsabilidades, **JPA/Hibernate** para la persistencia y conexión de la aplicación con la base de datos y **WebClient** en el microservicio Service-Pedido para comunicarse con los microservicios productores, a través del enriquecimiento de datos. Se realizaron las configuraciones en **application.properties** otorgándoles puertos de servidores que permitan la interacción entre microservicios.

Estructura interna aplicada por servicio:

- **Capa Model:** Utilicé anotaciones de Jakarta Persistence (@Entity, @Table @Id, @GeneratedValue, @Column, @Transient) para indicarle a Spring Boot que una clase de Java se transformará en una tabla de base de datos y para reducir el código repetitivo, y de Lombok (@Data, @AllArgsConstructor, @NoArgsConstructor) para generar automáticamente elementos como accedores (getters), mutadores (setters) y constructores durante la compilación. Utilicé anotaciones de la dependencia Validation como @NotNull, @NotBlank y @Min para la validación de datos obligatorios, evitando los campos vacíos en String (@NotBlank) e Integer (@NotNull) y determinar valores mínimos (@Min).
- **Capa Repository:** Implementé la interfaz que extiende JPA Repository para abstraer las operaciones CRUD (Create, Read, Update, Delete) y recurrir a operaciones de consultas (queries) SQL, principalmente en la tabla **Donut** y **Cientes**.
- **Capa Service:** Anotado con @Service, programé la lógica de negocio, validaciones y reglas operativas, se utilizó la anotación @Valid para validar que el objeto a crear no tenga campos nulos y vacíos.
- **Capa Controller:** Anotado con @RestController y @RequestMapping, programé los endpoints para las peticiones del cliente, usando GET (obtener información del objeto), POST (crear un objeto) y DELETE (borrar un objeto a partir de su ID). El PUT no se utilizó para los microservicios de Catálogo, Clientes y Pedido.

Microservicios desarrollados:

- **service-catalogo:** Desarrollé el microservicio encargado de la gestión del catálogo de donuts. Desarrollé los endpoints para listar todos los donuts creados, filtrar por clave primaria (ID) y categorías, realizar el conteo por categoría y la búsqueda de donuts por stock.
- **servicio-clientes:** Desarrollé el microservicio encargado de la gestión de los datos de los clientes y los endpoints para listar todos los clientes creados, filtrar por clave primaria (ID) y nombre del cliente y obtener el total de clientes creados.
- **service-pedido:** Desarrollé el núcleo transaccional del proyecto. Este microservicio se encarga del procesamiento de la orden de compra, mediante los datos del producto y del cliente. Realicé el enriquecimiento de datos a partir de la referencia lógica al microservicio **service-catalogo** para obtener la información de las tablas mediante la clave primaria de **Donut**.

IV. PRUEBAS DE INTEGRACIÓN EN POSTMAN

Me encargué de realizar las pruebas del proyecto completo en **Postman** mediante acuerdo con mi colaborador, con el fin de garantizar la correcta comunicación de los endpoints mediante el puerto 8080 de la API Gateway. En el proceso me encargué de encender las terminales, probar el programa y corregir mis códigos mal estructurados y de informar a mi colaborador para la corrección de los códigos de su parte del proyecto, evitando el retorno de estados HTTP como 400 Bad Request y 500 Internal Server Error. Se realizaron alrededor de 6 pruebas de API REST que se almacenaron en la base de datos.

V. CONCLUSIÓN

Mi aporte individual al proyecto permitió que el proyecto contara con una base de datos robusta y normalizada, una lógica de backend modular y desacoplada a través de tres microservicios altamente funcionales, como lo fueron los módulos encargados de la producción de información (service-catalogo y servicio-cliente) y el encargado de gestionar la compra (servicio-pedido) a partir de la información de los módulos anteriores, y una API testeada y documentada a nivel de endpoints. En conclusión, el proyecto cumple con los requisitos que se exigen para una arquitectura de microservicios desacoplada, como la modularidad, escalabilidad y desacoplamiento, garantizando la persistencia de los datos.

PARTE I – Benjamín González

I. INTRODUCCIÓN

El presente documento detalla mi contribución individual en el desarrollo del proyecto del sistema de Venta de Donuts (Sabores Felices Donutin). Mi rol en el proyecto se centró en el desarrollo backend, donde asumí la responsabilidad de diseñar e implementar dos microservicios críticos para el sistema. Asimismo, me encargué del desarrollo y la correcta integración de la API, asegurando la comunicación eficiente, la sincronización y el funcionamiento óptimo de todos los componentes de la arquitectura.

II. DESARROLLO BACKEND Y LÓGICA DE NEGOCIO EN SPRING BOOT

En la etapa de codificación, con mi colaborador nos dividimos los microservicios y desarrollé los microservicios receptores de información (Pagos y Logística). La construcción de los módulos se realizó con **Spring Boot**, empleando el patrón por capas (**Controller, Service, Repository**) para asegurar un diseño limpio y modular. Con el fin de lograr la interacción con los microservicios productores, se implementó **WebClient** en ambos microservicios. Se realizaron las configuraciones en **application.properties** otorgándoles puertos de servidores que permitan la interacción entre microservicios.

Estructura interna aplicada por servicio:

- **Capa Model:** El desarrollo de las entidades se optimizó mediante **Jakarta Persistence** para el mapeo de tablas en la base de datos, y **Lombok** para automatizar la creación de constructores, getters y setters, disminuyendo el código innecesario.

Adicionalmente se gestionó la consistencia de los datos mediante la API de **Validation**, implementando restricciones como **@NotBlank** en cadenas de texto, **@Min** en valores numéricos y **@NotNull** para asegurar el ingreso obligatorio tanto de datos enteros como datos Booleanos

- **Capa Repository:** La arquitectura de datos para los componentes de Pagos y Logística se consolidó mediante interfaces que extienden de **JpaRepository**, delegando en el framework el manejo automatizado de la persistencia para las entidades Pago, ComprobantePago y Despacho. Con el propósito de indexar la información de manera más eficiente, se definieron consultas derivadas en los repositorios principales de ambos microservicios. Específicamente mediante el método **FindByPedidoId**, el sistema es capaz de filtrar y retornar listas detalladas de pagos y despachos vinculados a un identificador de pedido único, garantizando un flujo de información integrado, rápido y desacoplado.
- **Capa Service:** En esta capa se programaron las reglas de negocio de los microservicios Pagos y Logística mediante componentes **@Service**. Al ser una arquitectura distribuida, se implementó **WebClient** en ambos módulos para conectarse con los microservicios productores. Esto permitió realizar el enriquecimiento de datos, logrando que cada servicio obtenga información externa de los pedidos en tiempo real para procesar correctamente transacciones y despachos.
- **Capa Controller:** En esta capa se desarrollaron los puntos de entrada de los microservicios Pagos y Logística mediante la anotación **@RestController** y **@RequestMapping**. Su función principal es exponer las API REST del sistema, interceptar las peticiones HTTP externas. Para ello se implementó **@Autowired** para la inyección automática de dependencias de la capa servicio, y se mapean los métodos de la API

utilizando **@PostMapping** para la creación de registros, **@GetMapping** para las consultas de información y **@PutMapping** para las actualizaciones. Todo esto validando los datos de entrada y retornando las respuestas en formato JSON con sus respectivos códigos de estado HTTP

III. IMPLEMENTACIÓN DE LA API GATEWAY

Se implementó una API Gateway para la comunicación de los microservicios, encapsulando la estructura interna de los 5 microservicios(service-catalogo, service-clientes, service-pedido, service-pago y service-logistica).

A través del API Gateway, se configuraron las reglas de enrutamiento dinámico. Cuando un cliente realiza una petición HTTP, el Gateway intercepta la solicitud y la redirige al microservicio correspondiente según la ruta. Esta implementación evita que el cliente o usuario deba conocer los puertos individuales de cada servidor, lo que centraliza el flujo de datos del proyecto.

IV. CORRECCIÓN DE ERRORES

Durante la fase de pruebas del backend se identificaron fallos en la lógica de persistencia y validación de datos que afectan la estabilidad de los microservicios, entre ellos se encontraban los siguientes errores:

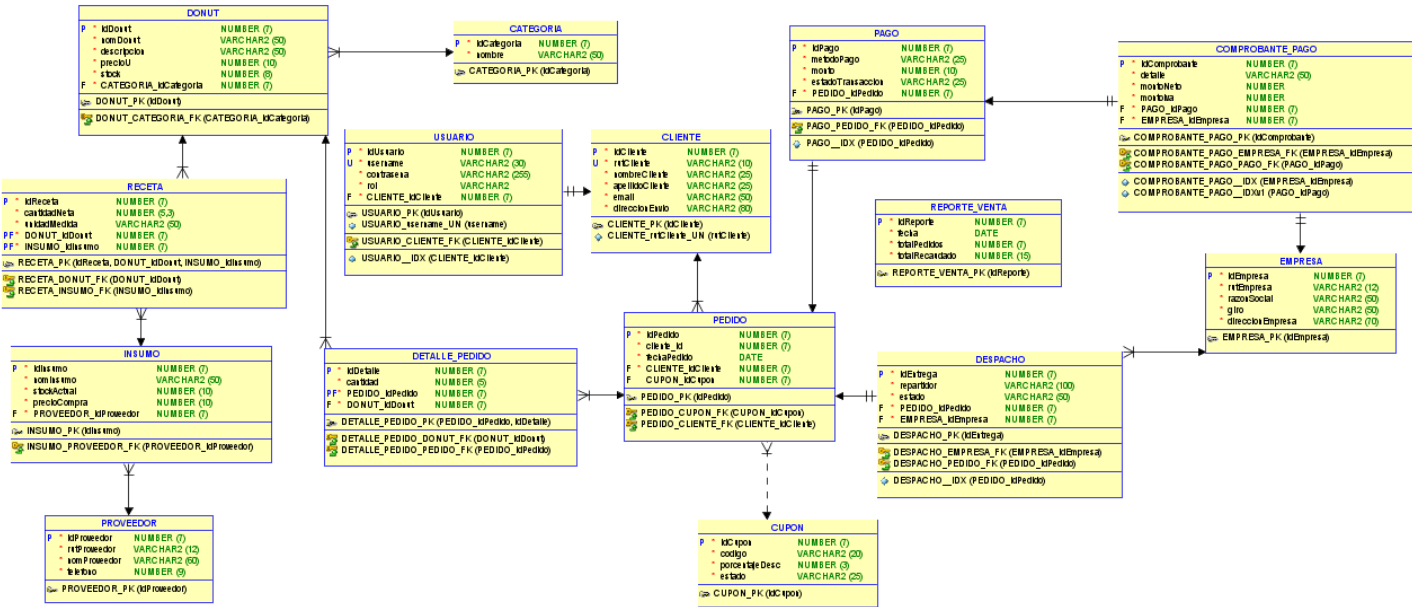
- **Corrección en la validación de tipos de datos:** Se detectaron inconsistencias al procesar valores obligatorios. Se corrigieron las entidades para aplicar estrictamente la anotación **@NotNull** en campos de tipo numérico (Integer) y lógicos (Boolean), asegurando que el flujo de datos no fallará con valores nulos en tiempo de ejecución y previniendo excepciones en la base de datos.
- **Depuración del flujo de persistencia:** Se corrigieron errores de mapeo relacional e inserción en las entidades de los módulos de Pagos y Logística, garantizando la correcta escritura de datos y optimizando el rendimiento de las consultas derivadas (FindByPedidoId)

V. CONCLUSIÓN

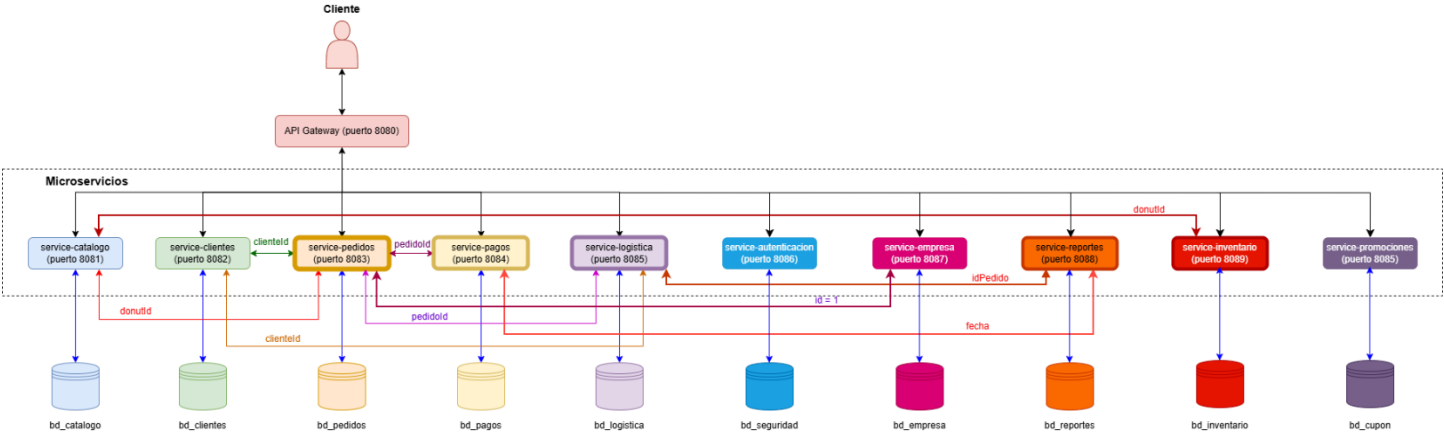
Mi participación en el proyecto de Venta de Donuts (Sabores Felices Donutin) termina de consolidar una arquitectura distribuida y escalable. Mi contribución individual se centró en dos pilares clave: la implementación integral de los microservicios Pago y Logistica bajo un patrón por capas y el diseño del API Gateway como punto único de entrada para articular los 5 microservicios del proyecto.

PARTE II

I. MODELO DE BASE DE DATOS



II. ARQUITECTURA DE MICROSERVICIOS



III. SERVICIO DE AUTENTICACIÓN

¿En qué consiste?

Es un microservicio independiente encargado de la verificación de la identidad de los usuarios y emisión de credenciales y diseñado para proteger los recursos de una arquitectura orientada a microservicios. Se encarga de gestionar los usuarios, los roles y emitir tokens JWT (JSON Web Tokens) para que otros servicios no tengan que gestionar usuarios y contraseñas. Cuando un usuario se autentica exitosamente, el servicio genera un token firmado digitalmente que contiene la identidad y los privilegios del usuario. Este token viaja en cada petición HTTP subsiguiente dentro de las cabeceras, permitiendo que cualquier microservicio verifique de forma independiente y matemática la legitimidad de la solicitud sin consultar una base de datos externa.

El token generado se compone estructuralmente de tres partes separadas por puntos:

1. Header (cabecera): Indica que es un JWT y usa HS256.
2. Payload (carga útil): Contiene el nombre de usuario, los roles, la fecha de creación y expiración.
3. Signature (firma): El sello de seguridad creado con tu token secreto.

¿Cómo se implementa la autenticación por token?

1. Base de datos

Se monta una base de datos independiente en MySQL para gestionar la persistencia de las credenciales de acceso:

- **usuarios:** Almacena el identificador único, nombre de usuario, correo y contraseña. La clave nunca se guarda en texto plano, sino encriptada mediante algoritmos seguros de hashing como BCrypt.
- **roles:** Contiene los permisos iniciales del ecosistema (ej: CLIENTE, GERENTE, PASTELERO, CONTADOR, REPARTIDOR).
- **usuario_roles:** Tabla intermedia que maneja la relación de muchos a muchos (@ManyToMany) entre usuarios y sus respectivos roles.

2. Microservicio service-auth

Este microservicio cuenta con dependencias claves como Spring Security y librerías de firma jjwt. La lógica está distribuida en las siguientes capas:

- **JwtService:** Es el encargado de construir el “pasaporte digital” (Token JWT) codificado en Base64 utilizando Jwts.builder(). Define el dueño, añade permisos como textos planos, establece tiempos de expiración de 2 horas y estampa una firma digital criptográfica (HS256) basada en una llave secreta.
- **AuthService:** Es la lógica de negocio. En el Registro toma la contraseña del JSON, la transforma en un hash indescifrable vía passwordEncoder.encode() y persiste al usuario. En el Login se busca al usuario en la base de datos, compara la clave ingresada contra el hash almacenado mediante passwordEncoder.matches(), y si coinciden mapea los roles a una lista de strings para retornar el JWT.
- **SecurityConfig:** Desactiva la protección CSRF al tratarse de una API REST e indica que toda ruta bajo el patrón /auth/ es pública, mientras que cualquier otra petición al microservicio requiere autenticación forzada.
- **AuthenticationController:** Expone los endpoints HTTP con arquitectura REST (POST) para procesar el registro y el inicio de sesión mapeando las solicitudes mediante DTOs.

3. API Gateway como filtro global de control (puerto 8080)

El API Gateway actúa como el “guardia de seguridad” de toda la arquitectura. Utilicé las mismas librerías y la misma firma secreta que el servicio de autenticación para validar las peticiones sin tocar las bases de datos externas.

- **AuthenticationFilter:** Intercepta de forma reactiva cada petición dirigida a las rutas configuradas. Su comportamiento sigue el siguiente flujo de validación.
 - a. Extrae la cabecera HTTP Authorization de la solicitud entrante.
 - b. Valida que la cabecera contenga datos y empiece obligatoriamente con el prefijo “Bearer”. Si no lo cumple corta el flujo con un error 401 Unauthorized.
 - c. Limpia el string para aislar el token puro y ejecuta Jwts.parserBuilder().setSigningKey(...).build().parseClaimsJws(token).
 - d. Si la firma no coincide o el token ya expiró cronológicamente, el bloque catch atrapa la excepción y retorna un error al cliente.
 - e. Si el token es auténtico y legal, da luz verde para que la petición continúe su viaje hacia el microservicio final de negocio.

IV. IMPLEMENTACIÓN POR CAPAS Y COMUNICACIÓN ENTRE MICROSERVICIOS

Implementación por capas

Capa Model: en esta capa se crean las entidades, se nombran las tablas y se establecen los atributos con sus datos y las validaciones, Hibernate se encarga de crear estas entidades de forma automática en la base de datos.

Donut:

```
service-catalogo > src > main > java > com > donutin > service_catalogo > model > J Donut.java > ...
1 package com.donutin.service_catalogo.model;
2
3 import io.swagger.v3.oas.annotations.media.Schema;
4 import jakarta.persistence.Column;
5 import jakarta.persistence.Entity;
6 import jakarta.persistence.GeneratedValue;
7 import jakarta.persistence.GenerationType;
8 import jakarta.persistence.Id;
9 import jakarta.persistence.JoinColumn;
10 import jakarta.persistence.ManyToOne;
11 import jakarta.persistence.Table;
12 import jakarta.validation.constraints.Min;
13 import jakarta.validation.constraints.NotBlank;
14 import jakarta.validation.constraints.NotNull;
15 import lombok.AllArgsConstructor;
16 import lombok.Data;
17 import lombok.NoArgsConstructor;
18
19 @Entity
20 @Table(name= "Donut")
21 @Data
22 @AllArgsConstructor
23 @NoArgsConstructor
24 @Schema(description = "Modelo que representa a una donut en el sistema")
25 public class Donut
26 {
27     @Id
28     @GeneratedValue(strategy = GenerationType.IDENTITY)
29     @Schema(description = "ID único autoincremental", example = "1")
30     private Long id;
31
32     @NotBlank(message = "El nombre no puede estar vacío")
33     @Column(nullable = false, length = 100)
34     @Schema(description = "Nombre de la donut", example = "Donut chocolate")
35     private String nombreDonut;
36
37     @NotBlank(message = "La descripción no puede estar vacía")
38     @Column(nullable = false, length = 100)
39     @Schema(description = "Descripción de la donut", example = "Donut cubierta de chocolate")
40     private String descripcion;
41
42     @NotNull(message = "El precio no puede ser nulo")
43     @Min(value = 1, message = "El precio debe ser mayor a 0")
44     @Column(nullable = false)
45     @Schema(description = "Precio entero de la donut sin signo peso", example = "2990")
46     private Integer precioUnitario;
47
48     @NotNull(message = "El stock no puede ser nulo")
49     @Min(value = 0, message = "El stock no puede ser negativo")
50     @Column(nullable = false)
51     @Schema(description = "Cantidad de productos guardados en la empresa", example = "50")
52     private Integer stock;
53
54     @NotNull(message = "La categoría no puede ser nula")
55     @ManyToOne
56     @JoinColumn(name = "categoria_id", nullable = false)
57     private Categoria categoria;
58 }
59
```

Categoría:

```
service-catalogo > src > main > java > com > donutin > service_catalogo > model > J Categoria.java > {} com.donutin.service_catalogo.model
1 package com.donutin.service_catalogo.model;
2
3 import jakarta.persistence.Entity;
4 import jakarta.persistence.GeneratedValue;
5 import jakarta.persistence.GenerationType;
6 import jakarta.persistence.Id;
7 import jakarta.persistence.Table;
8 import lombok.AllArgsConstructor;
9 import lombok.Data;
10 import lombok.NoArgsConstructor;
11
12 @Entity
13 @Table(name= "categoria")
14 @Data
15 @AllArgsConstructor
16 @NoArgsConstructor
17 public class Categoria
18 {
19     @Id
20     @GeneratedValue(strategy = GenerationType.IDENTITY)
21     private Long idCategoria;
22     private String nombre; //Ej: Vegano, sin azúcar, premium
23 }
```

Capa Repository: se encarga de abstraer el almacenamiento y recuperación de datos. En el proyecto se realizaron consultas (query) para abstraer el CRUD.

Donut:

```
service-catalogo > src > main > java > com > donutin > service_catalogo > repository > J DonutRepository.java >
1 package com.donutin.service_catalogo.repository;
2
3 import java.util.List;
4
5 import org.springframework.data.jpa.repository.JpaRepository;
6 import org.springframework.data.jpa.repository.Query;
7
8 import com.donutin.service_catalogo.model.Donut;
9
10 public interface DonutRepository extends JpaRepository<Donut, Long>
11 {
12     Donut findByNombreDonut(String nombre);
13     Donut findByNombreDonutIgnoreCase(String nombre);
14     @Query("""
15         SELECT d
16         FROM Donut d
17         WHERE d.categoria.idCategoria = :idCategoria
18         """)
19     List<Donut> buscarDonasPorCategoria(Long idCategoria);
20
21     @Query("""
22         SELECT d.categoria.nombre as nomCategoria,
23         COUNT(d) as cantidad
24         FROM Donut d
25         GROUP BY d.categoria.nombre
26         """)
27     List<Object[]> conteoPorNombreCategoria();
28
29     @Query("""
30         SELECT d
31         FROM Donut d
32         WHERE d.stock = :stock
33         """)
34     List<Donut> buscarDonasPorStock(Integer stock);
35 }
36
```

Categoría:

```
service-catalogo > src > main > java > com > donutin > service_catalogo > repository > J CategoriaRepository.java >
1 package com.donutin.service_catalogo.repository;
2
3 import org.springframework.data.jpa.repository.JpaRepository;
4
5 import com.donutin.service_catalogo.model.Categoria;
6
7
8 public interface CategoriaRepository extends JpaRepository<Categoria, Long>
9 {
10     Categoria findByNombre(String nombre);
11     Categoria findByNombreIgnoreCase(String nombre);
12 }
13
```


Capa Service: esta capa se encarga de la lógica de negocio, a partir de la abstracción de los datos en el Repository.

```

service-catalogo > src > main > java > com > donut > service_catalogo > service > J DonutService.java > Language Support for Java(TM) by Red Hat > ( ) com.donut.service_catalogo.service
1 package com.donut.service_catalogo.service;
2
3 import java.util.List;
4 import java.util.Optional;
5
6 import org.springframework.stereotype.Service;
7 import org.springframework.validation.annotation.Validated;
8
9 import com.donut.service_catalogo.model.Categoria;
10 import com.donut.service_catalogo.model.Donut;
11 import com.donut.service_catalogo.repository.CategoriaRepository;
12 import com.donut.service_catalogo.repository.DonutRepository;
13
14 import jakarta.transaction.Transactional;
15 import jakarta.validation.Valid;
16
17 @Service
18 @Validated
19 public class DonutService
20 {
21     private final DonutRepository donutRepository;
22     private final CategoriaRepository categoriaRepository;
23
24     public DonutService(DonutRepository donutRepository, CategoriaRepository categoriaRepository)
25     {
26         this.donutRepository = donutRepository;
27         this.categoriaRepository = categoriaRepository;
28     }
29
30     public List<Donut> listarTodos()
31     {
32         return donutRepository.findAll();
33     }
34     public Optional<Donut> buscarPorId(Long id)
35     {
36         return donutRepository.findById(id);
37     }
38     @Transactional
39     public Donut guardar(@Valid Donut donut)
40     {
41         if(donut.getCategoria()!=null && donut.getCategoria().getIdCategoria()!=null)
42         {
43             Categoria categoriaCompleta = categoriaRepository.findById(donut.getCategoria().getIdCategoria()).orElse( null);
44             donut.setCategoria(categoriaCompleta);
45         }
46         return donutRepository.save(donut);
47     }
48     public void eliminar(Long id)
49     {
50         donutRepository.deleteById(id);
51     }
52     public List<Donut> buscarPorCategoria(Long idCategoria)
53     {
54         return donutRepository.buscarDonasPorCategoria(idCategoria);
55     }
56     public List<Object[]> conteoPorNombreCategoria()
57     {
58         return donutRepository.conteoPorNombreCategoria();
59     }
60     public List<Donut> buscarDonasPorStock(Integer stock)
61     {
62         return donutRepository.buscarDonasPorStock(stock);
63     }
64 }
65
```

Capa Controller: esta capa se encarga de recibir las solicitudes del usuario, procesarlas enviándolas a la lógica de negocio y devolver una respuesta, en el caso del proyecto, se devuelven en formato JSON en Postman.

Donut:

```

service-catalogo > src > main > java > com > donut > service_catalogo > controller > J DonutController.java > Language Support for Java(TM) by Red Hat > ( ) com.donut.service_catalogo.controller
1 package com.donut.service_catalogo.controller;
2
3 import java.util.List;
4
5 import org.springframework.http.HttpStatus;
6 import org.springframework.http.ResponseEntity;
7 import org.springframework.web.bind.annotation.CrossOrigin;
8 import org.springframework.web.bind.annotation.DeleteMapping;
9 import org.springframework.web.bind.annotation.GetMapping;
10 import org.springframework.web.bind.annotation.PathVariable;
11 import org.springframework.web.bind.annotation.PostMapping;
12 import org.springframework.web.bind.annotation.RequestBody;
13 import org.springframework.web.bind.annotation.RequestMapping;
14 import org.springframework.web.bind.annotation.RestController;
15
16 import com.donut.service_catalogo.exception.ErrorResponse;
17 import com.donut.service_catalogo.model.Donut;
18 import com.donut.service_catalogo.service.DonutService;
19
20 import io.swagger.v3.oas.annotations.Operation;
21 import io.swagger.v3.oas.annotations.media.Content;
22 import io.swagger.v3.oas.annotations.media.Schema;
23 import io.swagger.v3.oas.annotations.responses.ApiResponse;
24 import io.swagger.v3.oas.annotations.responses.ApiResponses;
25 import io.swagger.v3.oas.annotations.tags.Tag;
26 import jakarta.validation.Valid;
27
28 @RestController
29 @RequestMapping("/api/v1/catalogo/donuts")
30 @Tag(name = "Donuts", description = "Operaciones relacionadas con la gestión de las donuts")
31 @CrossOrigin(origins = "*") //Permite que Swagger lo llame desde cualquier puerto
32 public class DonutController
33 {
34     private final DonutService donutService;
35
36     public DonutController(DonutService donutService)
37     {
38         this.donutService = donutService;
39     }
40
41     @Operation(summary = "Obtener todas las donuts", description = "Retorna una lista completa de donuts registradas")
42     @GetMapping
43     public List<Donut> listar()
44     {
45         return donutService.listarTodos();
46     }
47     @Operation(summary = "Obtener por ID")
48     @ApiResponses(value = {
49         @ApiResponse(responseCode = "200", description = "Donut encontrada"),
50         @ApiResponse(responseCode = "404", description = "No existe la donut que busca", content = @Content(schema = @Schema(implementation = ErrorResponse.class)))
51     })
52     @GetMapping("/{id}")
53     public ResponseEntity<Donut> obtenerPorId(@PathVariable Long id)
54     {
55         return donutService.buscarPorId(id)
56             .map(ResponseEntity::ok)
57             .orElseThrow(() -> new RuntimeException(message: "Donut no encontrada"));
58     }
59     @GetMapping("/categoria/{id}")
60     public ResponseEntity<List<Donut>> buscarPorCategoria(@PathVariable Long id)
61     {
62         List<Donut> donut = donutService.buscarPorCategoria(id);
63         if(donut.isEmpty())
64         {
65             return ResponseEntity.noContent().build();
66         }
67     }
68 }
69
```


Comunicación del microservicio service-catalogo

GET

GET http://localhost:8080/api/

+

...

http://localhost:8080/api/v1/catalogo/donuts

Save

GET

http://localhost:8080/api/v1/catalogo/donuts

Send

Params

Authorization

Headers (9)

Body

Pre-request Script

Tests

Settings

Cookies

Body

Cookies

Headers (9)

Test Results

Status: 200 OK

Time: 127 ms

Size: 1.57 KB

Save Response

Pretty

Raw

Preview

Visualize

JSON

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

{

"id": 2,

"nombreDonut": "Brocheta Donutin",

"descripcion": "Brocheta Donutin con 4 deliciosas mini donuts",

"precioUnitario": 1000,

"stock": 40,

"categoria": {

"idCategoria": 1,

"nombre": "Vegano"

}

},

{

"id": 3,

"nombreDonut": "Vaso Donutin Peque",

"descripcion": "Vaso Donutin con 8 deliciosas mini donuts",

"precioUnitario": 2000,

"stock": 100,

"categoria": {

"idCategoria": 2,

"nombre": "Sin lactosa"

}

},

{

"id": 4,

"nombreDonut": "Vaso Donutin Max",

GET(id)

GET http://localhost:8080/api/

+

...

http://localhost:8080/api/v1/catalogo/donuts/4

Save

GET

http://localhost:8080/api/v1/catalogo/donuts/4

Send

Params

Authorization

Headers (9)

Body

Pre-request Script

Tests

Settings

Cookies

Body

Cookies

Headers (9)

Test Results

Status: 200 OK

Time: 70 ms

Size: 480 B

Save Response

Pretty

Raw

Preview

Visualize

JSON

1

2

3

4

5

6

7

8

9

10

11

{

"id": 4,

"nombreDonut": "Vaso Donutin Max",

"descripcion": "Vaso Donutin con 12 deliciosas mini donuts",

"precioUnitario": 3000,

"stock": 20,

"categoria": {

"idCategoria": 3,

"nombre": "Sin azúcar"

}

}

POST

POST http://localhost:8080/api/

+

...

http://localhost:8080/api/v1/catalogo/donuts

Save

POST

http://localhost:8080/api/v1/catalogo/donuts

Send

Params

Authorization

Headers (9)

Body

Pre-request Script

Tests

Settings

Cookies

Body

Cookies

Headers (9)

Test Results

Status: 201 Created

Time: 258 ms

Size: 480 B

Save Response

Pretty

Raw

Preview

Visualize

JSON

1

2

3

4

5

6

7

8

9

10

11

{

"id": 9,

"nombreDonut": "Vaso Donutin Max",

"descripcion": "Vaso Donutin con 12 deliciosas mini donuts",

"precioUnitario": 3000,

"stock": 18,

"categoria": {

"idCategoria": 8,

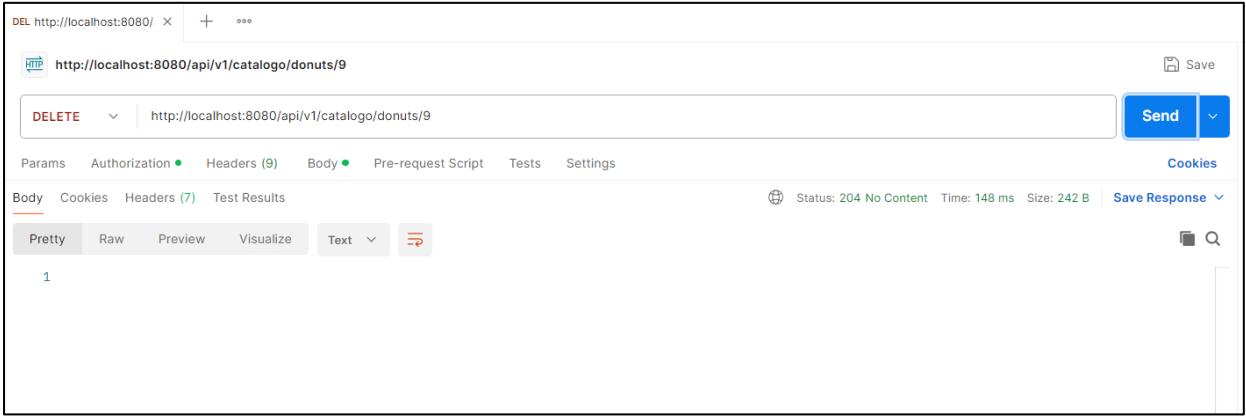
"nombre": "Vegano"

}

}

DELETE (id)

Cuando se elimina una Donut, aparece el status **204 No Content**, que significa que la información del donut fue eliminada de la base de datos.



Este microservicio no cuenta con PUT, este endpoint REST solo está presente en el microservicio service-logistica para actualizar el estado del Reparto del pedido.

¿Cómo se implementa la comunicación entre microservicios?

Aunque el **service-catalogo** no tenga un cliente HTTP (WebClient) para consumir a otros, sigue siendo parte fundamental de la comunicación del sistema adoptando un rol pasivo: **es un servidor de recursos**.

1. Cuando se manda un GET desde Postman a servicio-pedido para ver un pedido de donuts.
2. El servicio-pedido ve que el donut comprado tiene el ID 3, por ejemplo.
3. El WebClient del servicio-pedido hace un viaje por red hacia el puerto del catálogo: <http://localhost:8081/api/v1/catalogo/donuts/5>.
4. El service-catalogo recibe la petición en su DonutController, procesa mediante su DonutService, saca la información de su base de datos y la devuelve como JSON.

V. SWAGGER

¿Qué es? ¿En qué consiste?

Swagger es una herramienta para documentar APIs REST. Ofrece documentación interactiva y visual que ayuda a los desarrolladores y a los usuarios de la API a entender mejor cómo funciona la API, qué solicitudes se pueden realizar, qué parámetros se deben pasar y qué respuestas esperar.

¿Cómo se implementa?

En el ecosistema de Spring Boot, la librería estándar para implementar Swagger es SpringDoc OpenAPI. La implementación en un entorno de microservicios sigue los siguientes pasos:

- **Agregar la dependencia:** Se incluye el starter de springdoc-openapi-starter-webmvc-ui en el pom.xml.
- **Configurar Bean de OpenAPI:** Se crea una clase de configuración en Java para definir el título, versión y descripción de la API.
- **Anotar los Controladores:** Se usan anotaciones como @Operation, @ApiResponse y @Tag para enriquecer la documentación directamente en el código.
- **Centralización (Opcional pero recomendado):** En una arquitectura de microservicios, se suele configurar el **API Gateway** para que consuma las definiciones de Swagger de todos los microservicios y los muestre en una sola interfaz unificada.

service-autenticacion

GET

Autenticación

Endpoints para registro e inicio de sesión de usuarios

POST

/api/v1/auth/regar

Registrar nuevo usuario

Guarda el usuario con contraseña encriptada

Parameters

No parameters

Request body

required

application/json

```
{  "nombreUsuario": "Daniel Moreno",  "contrasena": "3436",  "correo": "danimoreno@gmail.com",  "roles": [    "CLIENTE"  ]}
```

Responses

Curl

```
curl -X 'POST' \  'http://localhost:8086/api/v1/auth/regar' \  -H 'accept: */*' \  -H 'Content-Type: application/json' \  -d '{  "nombreUsuario": "Daniel Moreno",  "contrasena": "3436",  "correo": "danimoreno@gmail.com",  "roles": ["CLIENTE"]  }'
```

Request URL

http://localhost:8086/api/v1/auth/regar

Server response

Code

Details

200

Response body

Usuario registrado con éxito

Download

POST

POST

/api/v1/auth/login

Iniciar sesión

Retorna el Token JWT si las credenciales son válidas

Parameters

Cancel

Reset

No parameters

Request body

required

application/json

```
{  "nombreUsuario": "Daniel Moreno",  "contraseña": "3456",  "correo": "danimoreno@gmail.com",  "roles": ["GERENTE"]}
```

Execute

Clear

Responses

Curl

```
curl -X "POST" \
    http://localhost:8086/api/v1/auth/login' \
-H 'accept: */*' \
-H 'Content-Type: application/json' \
-d '{
  "nombreUsuario": "Daniel Moreno",
  "contrasena": "3456",
  "correo": "danimoreno@gmail.com",
  "roles": ["GERENTE"]
}'
```

Request URL

http://localhost:8086/api/v1/auth/login

Server response

Code	Details
200	Response body yJhbGciOiJIUzI1NiIsInR5cGU6ImF1dG8iLCJpcyZvbnQ6ZXNpdjEwS2RVSURUSURScjdlCjPVPXQ1OjE3ODIwMzIxZWJsImV4cCI6MTc4MjA3NTkzMH0uYXBvdW50fB1cc1uqxKIOXSdyC1r_zr6BF7bohHmDcw

service-catalogo

Donut:
GET

DonutsOperaciones relacionadas con la gestión de los donuts

GET/api/v1/catalogo/donutsObtener todas las donuts

Retorna una lista completa de donuts registradas

Parameters

Cancel

No parameters

ExecuteClear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8081/api/v1/catalogo/donuts' \
  -H 'accept: */*'

```

Request URL

http://localhost:8081/api/v1/catalogo/donuts

Server response

CodeDetails

200

Response body

```
{
  "id": 2,
  "nombreDonut": "Brocheta Donutin",
  "descripcion": "Brocheta Donutin con 4 deliciosas mini donuts",
  "precioUnitario": 1000,
  "stock": 40,
  "categoria": {
    "idCategoria": 3,
    "nombre": "Vegano"
  }
},
{
  "id": 3,
  "nombreDonut": "Vaso Donutin Peque",
  "descripcion": "Vaso Donutin con 8 deliciosas mini donuts",
  "precioUnitario": 2000,
  "stock": 100,
  "categoria": {
    "idCategoria": 3,
    "nombre": "Sin lactosa"
  }
},
{
  "id": 4,
  "nombreDonut": "Vaso Donutin Ma",
  "descripcion": "Vaso Donutin con 12 deliciosas mini donuts",
  "precioUnitario": 3000,

```

Download

POST/api/v1/catalogo/donutsCrear una nueva donut

Guardar una donut en la base de datos bd_catalogo

Parameters

CancelReset

No parameters

Request body

requiredapplication/json

```
{
  "nombreDonut": "Brocheta Donutin",
  "descripcion": "Brocheta Donutin con 4 deliciosas mini donuts",
  "precioUnitario": 1000,
  "stock": 30,
  "categoria": {
    "idCategoria": 9
  }
}

```

ExecuteClear

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8081/api/v1/catalogo/donuts' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d {
    "nombreDonut": "Brocheta Donutin",
    "descripcion": "Brocheta Donutin con 4 deliciosas mini donuts",
    "precioUnitario": 1000,
    "stock": 30,
    "categoria": {
      "idCategoria": 9
    }
  }
}

```

Request URL

http://localhost:8081/api/v1/catalogo/donuts

Server response

CodeDetails

201

UndocumentedResponse body

```
{
  "id": 10,
  "nombreDonut": "Brocheta Donutin",
  "descripcion": "Brocheta Donutin con 4 deliciosas mini donuts",
  "precioUnitario": 1000,
  "stock": 30,
  "categoria": {
    "idCategoria": 9,
    "nombre": "Vegano"
  }
}

```

Download

GET (conteo-categoria)

GET /api/v1/catalogo/donuts/conteo-categoria

Parameters

No parameters

ExecuteClear

Responses

Curl

```
curl -X 'GET' \
'http://localhost:8081/api/v1/catalogo/donuts/conteo-categoria' \
-H 'accept: */*'

```

Request URL

```
http://localhost:8081/api/v1/catalogo/donuts/conteo-categoria

```

Server response

CodeDetails

200

Response body

```
{
  "Premium",
  1,
  "Sin azucar",
  "Sin gluten",
  "Sin lactosa",
  "Vegano",
  1
}

```

Response headers

```
content-type: application/json

```

Responses

CodeDescriptionLinks

GET (id)

GET /api/v1/catalogo/donuts/{id} Otener por ID

Parameters

NameDescription

id * required

integer(\$int64)

(path)

5

ExecuteClear

Responses

Curl

```
curl -X 'GET' \
'http://localhost:8081/api/v1/catalogo/donuts/5' \
-H 'accept: */*'

```

Request URL

```
http://localhost:8081/api/v1/catalogo/donuts/5

```

Server response

CodeDetails

200

Response body

```
{
  "id": 5,
  "nombreDonut": "Caja Donutin Premium",
  "descripcion": "Caja de 16 deliciosos donutines m\u00e1s 3 cucharitas",
  "precioUnitario": 6500,
  "stock": 100,
  "categoria": {
    "idCategoria": 1,
    "nombre": "Premium"
  }
}

```

Response headers

```
content-type: application/json

```

Categoria:

GET (id)

categoria-controller

GET /api/v1/catalogo/tipos

Parameters

No parameters

ExecuteClear

Responses

Curl

```
curl -X 'GET' \
'http://localhost:8081/api/v1/catalogo/tipos' \
-H 'accept: */*'

```

Request URL

```
http://localhost:8081/api/v1/catalogo/tipos

```

Server response

CodeDetails

200

Response body

```
{
  "idCategoria": 1,
  "nombre": "Vegano"
},
{
  "idCategoria": 2,
  "nombre": "Sin lactosa"
},
{
  "idCategoria": 3,
  "nombre": "Sin az\u00facar"
},
{
  "idCategoria": 4,
  "nombre": "Premium"
},
{
  "idCategoria": 5,
  "nombre": "Vegano"
},
{
  "idCategoria": 6,
  "nombre": "Sin gluten"
},
{
  "idCategoria": 7,
  "nombre": "Vegano"
}

```

POST

categoria-controller

GET /api/v1/catalogo/tipos

POST /api/v1/catalogo/tipos

Parameters

No parameters

Request body required

application/json

{
 "nombre": "Vegand"
}

Execute

Responses

Curl

```
curl -X 'POST' \
  http://localhost:8081/api/v1/catalogo/tipos' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{  
  "nombre": "Vegano"  
}'
```

Request URL

http://localhost:8081/api/v1/catalogo/tipos

Server response

Code	Details
200	Response body <pre>{ "idCategoria": 0, "nombre": "Vegano" }</pre> Response headers content-type: application/json

Responses

Code	Description	Links
200	OK	No links

Media type

"*/*"

Controls Accept header

service-clientes

POST

Cientes

Operaciones relacionadas con la gestión de clientes

GET

/api/v1/clientes

Obtener todos los clientes

POST

/api/v1/clientes

Crear un nuevo cliente

Guarda un cliente en la base de datos bd_clientes

Parameters

Cancel

Reset

No parameters

Request body

required

application/json

{
 "rutCliente": "18543987-6",
 "nombreCliente": "Juan Pablo",
 "apellidoCliente": "Domínguez Cáceres",
 "email": "jpadomacaceres@gmail.com",
 "direccion": "Avenida Las Lilas 123"
}

Execute

Responses

Curl

curl -X 'POST' \
 'http://localhost:8082/api/v1/clientes' \
 -H 'accept: */*' \
 -H 'Content-Type: application/json' \
 -d '{
 "rutCliente": "18543987-6",
 "nombreCliente": "Juan Pablo",
 "apellidoCliente": "Domínguez Cáceres",
 "email": "jpadomacaceres@gmail.com",
 "direccion": "Avenida Las Lilas 123"
 }'
}

Request URL

http://localhost:8082/api/v1/clientes

Server response

Code

Details

201

undocumented

Response body

{
 "idCliente": 1,
 "rutCliente": "18543987-6",
 "nombreCliente": "Juan Pablo",
 "apellidoCliente": "Domínguez Cáceres",
 "email": "jpadomacaceres@gmail.com",
 "direccion": "Avenida Las Lilas 123"
}

Response headers

content-type: application/json

GET (total)

GET

/api/v1/clientes/total

Parameters

Cancel

No parameters

Execute

Clear

Responses

Curl

curl -X 'GET' \
 'http://localhost:8082/api/v1/clientes/total' \
 -H 'accept: */*' \
}

Request URL

http://localhost:8082/api/v1/clientes/total

Server response

Code

Details

200

Response body

{
}

Download

GET

GET

/api/v1/clientes

Obtener todos los clientes

Retorna una lista completa de clientes registrados

Parameters

Cancel

No parameters

Execute

Clear

Responses

Curl

curl -X 'GET' \
 'http://localhost:8082/api/v1/clientes' \
 -H 'accept: */*' \
}

Request URL

http://localhost:8082/api/v1/clientes

Server response

Code

Details

200

Response body

{
 "idCliente": 1,
 "rutCliente": "12345678-9",
 "nombreCliente": "Juan",
 "apellidoCliente": "Pérez",
 "email": "juanp@gmail.com",
 "direccion": "Avda. República 4350"
},
 {
 "idCliente": 2,
 "rutCliente": "9876543-2",
 "nombreCliente": "Ana",
 "apellidoCliente": "Gómez",
 "email": "ana@gmail.com",
 "direccion": "Calle Camarero 640"
},
 {
 "idCliente": 3,
 "rutCliente": "8901234-5",
 "nombreCliente": "María",
 "apellidoCliente": "López",
 "email": "maria@gmail.com",
 "direccion": "Avda. España Blanca"
},
 {
 "idCliente": 4,
 "rutCliente": "11223344-8",
 "nombreCliente": "Manuel",
 "apellidoCliente": "García"
}

Download

VI. TESTING

¿En qué consiste? ¿Por qué es importante testearlas?

El testing en Spring Boot consiste en verificar que el código funcione correctamente mediante pruebas automatizadas (unitarias e de integración). El testing es importante porque garantiza la calidad, estabilidad y mantenibilidad del software a lo largo del tiempo.

¿En qué consisten las pruebas unitarias (AAA)?

Son el primer nivel de pruebas. Se centran en la unidad más pequeña de código de forma aislada (una función, un método o una clase).

- Objetivo: Verificar que una lógica específica funcione correctamente por sí sola.
- Aislamiento: Es clave. Si la función depende de una base de datos o una API externa, usamos "dobles" (mocks o stubs) para simular esas respuestas.
- Velocidad: Son extremadamente rápidas (milisegundos).

Implementar pruebas unitarias es fundamental para asegurar que cada "pieza" del proyecto Donutin (servicios y lógica de negocio) funcione correctamente de forma aislada, sin depender de que la base de datos o el otro microservicio estén encendidos.

En Spring Boot, usamos JUnit 5 y Mockito. Mockito es la herramienta que nos permite "simular" el comportamiento de los repositorios o de WebClient.

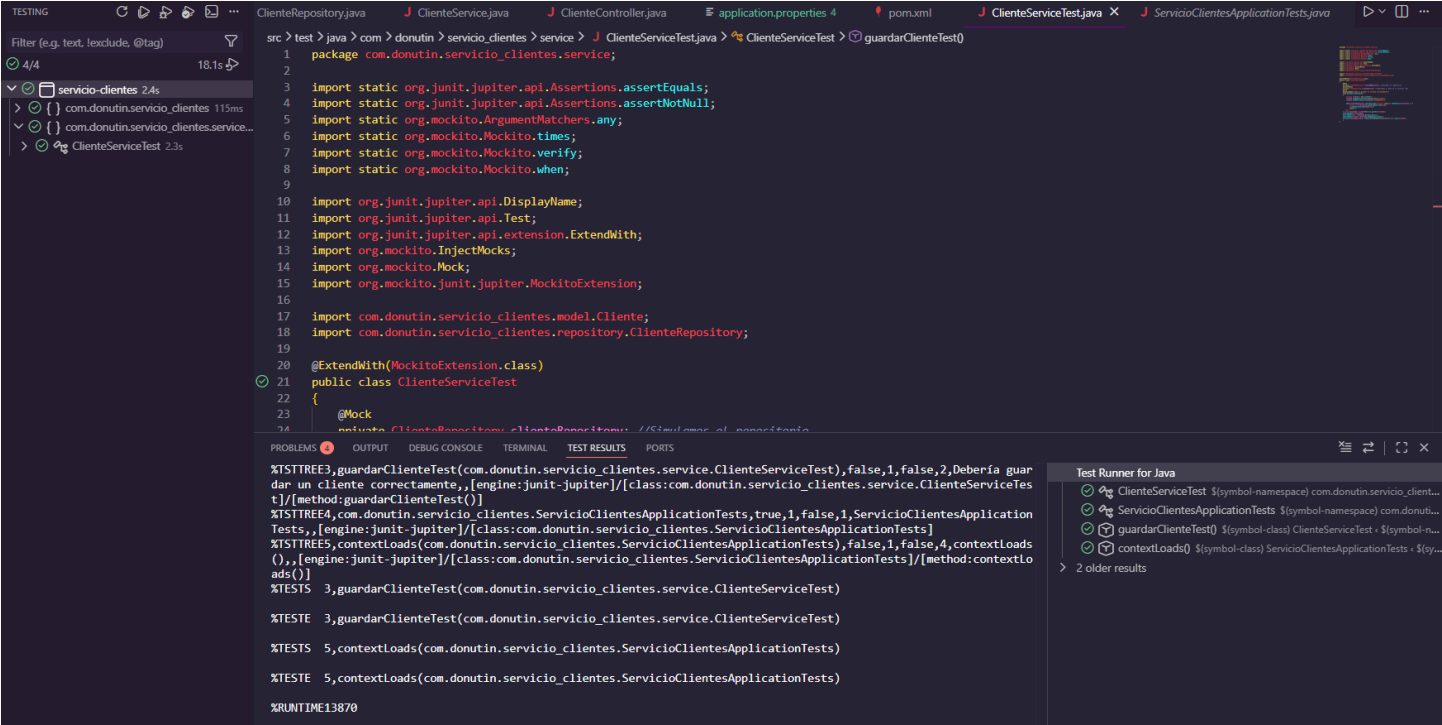
PRUEBAS UNITARIAS DEL PROYECTO

servicio-clientes

En esta prueba unitaria simulamos que cuando el repositorio ClienteRepository guarde cualquier cliente, retorne el mismo con ID 1, para evitar usar una base de datos real. En la preparación usamos solo el nombre y el RUT del cliente.

```
src > test > java > com > donutin > servicio_clientes > service > J ClienteServiceTest.java > ClienteServiceTest > guardarClienteTest()
1 package com.donutin.servicio_clientes.service;
2
3 import static org.junit.jupiter.api.Assertions.assertEquals;
4 import static org.junit.jupiter.api.Assertions.assertNotNull;
5 import static org.mockito.ArgumentMatchers.any;
6 import static org.mockito.Mockito.times;
7 import static org.mockito.Mockito.verify;
8 import static org.mockito.Mockito.when;
9
10 import org.junit.jupiter.api.DisplayName;
11 import org.junit.jupiter.api.Test;
12 import org.junit.jupiter.api.extension.ExtendWith;
13 import org.mockito.InjectMocks;
14 import org.mockito.Mock;
15 import org.mockito.junit.jupiter.MockitoExtension;
16
17 import com.donutin.servicio_clientes.model.Cliente;
18 import com.donutin.servicio_clientes.repository.ClienteRepository;
19
20 @ExtendWith(MockitoExtension.class)
21 public class ClienteServiceTest
22 {
23     @Mock
24     private ClienteRepository clienteRepository; //Simulamos el repositorio
25     @InjectMocks
26     private ClienteService clienteService; //Inyectamos el mock en el servicio real
27     @Test
28     @DisplayName("Debería guardar un cliente correctamente")
29     void guardarClienteTest()
30     {
31         Cliente cliente = new Cliente();
32         cliente.setNombreCliente(nombreCliente: "Juan");
33         cliente.setRutCliente(rutCliente: "12345678-9");
34
35         when(clienteRepository.save(any(type: Cliente.class))).thenAnswer(invocation -> {
36             Cliente c = invocation.getArgument(index: 0);
37             c.setIdCliente(idCliente: 1);
38             return c;
39         });
40         Cliente resultado = clienteService.guardar(cliente);
41         assertNotNull(resultado);
42         assertEquals(1, resultado.getIdCliente());
43         assertEquals("Juan", resultado.getNombreCliente());
44         verify(clienteRepository, times(wantedNumberOfInvocations: 1)).save(cliente);
45     }
46 }
47
```

Resultado del testeo al ClienteService

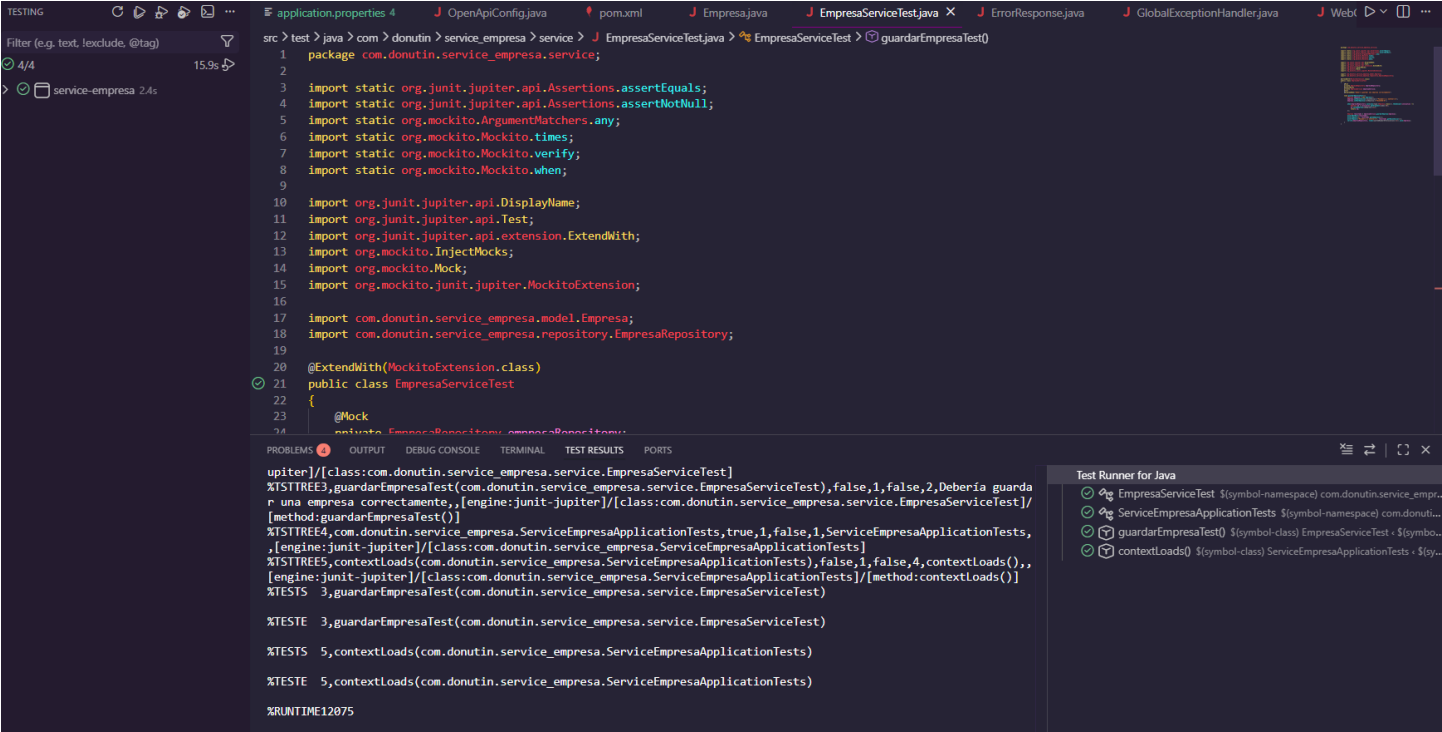


service-empresa

En esta prueba unitaria simulamos que cuando el repositorio EmpresaRepository guarde cualquier empresa, retorne el mismo con ID 1, para evitar usar una base de datos real. En la preparación usamos la razón social y el RUT de la empresa.

```
src > test > java > com > donutin > service_empresa > service > J EmpresaServiceTest.java > EmpresaServiceTest > guardarEmpresaTest()
1 package com.donutin.service_empresa.service;
2
3 import static org.junit.jupiter.api.Assertions.assertEquals;
4 import static org.junit.jupiter.api.Assertions.assertNotNull;
5 import static org.mockito.ArgumentMatchers.any;
6 import static org.mockito.Mockito.times;
7 import static org.mockito.Mockito.verify;
8 import static org.mockito.Mockito.when;
9
10 import org.junit.jupiter.api.DisplayName;
11 import org.junit.jupiter.api.Test;
12 import org.junit.jupiter.api.extension.ExtendWith;
13 import org.mockito.InjectMocks;
14 import org.mockito.Mock;
15 import org.mockito.junit.jupiter.MockitoExtension;
16
17 import com.donutin.service_empresa.model.Empresa;
18 import com.donutin.service_empresa.repository.EmpresaRepository;
19
20 @ExtendWith(MockitoExtension.class)
21 public class EmpresaServiceTest
22 {
23     @Mock
24     private EmpresaRepository empresaRepository;
25     @InjectMocks
26     private EmpresaService empresaService;
27     @Test
28     @DisplayName("Debería guardar una empresa correctamente")
29
30     void guardarEmpresaTest(){
31         Empresa empresa = new Empresa();
32         empresa.setRazonSocial(razonSocial: "Panadería y Cafetería");
33         empresa.setRutEmpresa(rutEmpresa: "75123456-8");
34
35         when(empresaRepository.save(any(type: Empresa.class))).thenAnswer(invocation ->{
36             Empresa em = invocation.getArgument(index: 0);
37             em.setIdEmpresa(idEmpresa: 1L);
38             return em;
39         });
40
41         Empresa resultado = empresaService.guardarEmpresa(empresa);
42         assertNotNull(resultado);
43         assertEquals(1L, resultado.getIdEmpresa());
44         assertEquals("Panadería y Cafetería", resultado.getRazonSocial());
45         verify(empresaRepository, times(wantedNumberOfInvocations: 1)).save(empresa);
46     }
47 }
48
```

Resultado del testeo a EmpresaService:



service-pedido (con WebClient)

Testear el GestionService de service-pedido es un desafío porque usa WebClient. Para no hacer una petición real al otro microservicio (service-catalogo), simulamos la respuesta.



```
67         when(responseSpec.bodyToMono(Object.class))
68             .thenReturn(Mono.just(donutMock)) // Primera Llamada: Donut
69             .thenReturn(Mono.just(empresaMock)); // Segunda Llamada: Empresa
70
71         //Ejecución
72         DetallePedido resultado = gestionService.obtenerDetallePedidoCompleto(pedidoDetalleId);
73
74         assertNotNull(resultado, message: "El detalle del pedido no puede estar nulo");
75         assertNotNull(resultado.getDatosDonuts(), message: "Los datos de la donut deben estar presentes");
76
77         PedidoDTO datosRetorno = (PedidoDTO) resultado.getDatosDonuts();
78         //Verificar datos del donut
79         assertEquals(expected: "Donut Crema", datosRetorno.getNombreDonut());
80         assertEquals(donutId, datosRetorno.getId());
81
82         //Verificar datos de La empresa
83         assertNotNull(resultado.getDatosEmpresa(), message: "Los datos de la empresa deben estar presentes");
84         assertEquals(expected: "Donutin Feliz", resultado.getDatosEmpresa());
85
86         //Verificamos que se Llamó al repositorio
87         verify(detallePedidoRepository, times(wantedNumberOfInvocations: 1)).findById(pedidoDetalleId);
88     }
89 }
90
```

Resultado del testing a GestionService:

TESTING

Filter (e.g. text, exclude, @tag)

4/4

servicio-pedido 2.5s

com.donutin.servicio_pedido 620ms

com.donutin.servicio_pedido.service... 1.9s

GestionServiceTest 1.9s

obtenerPedidoCompletoTest() 1.9s

src > test > java > com > donutin > servicio_pedido > service > J GestionServiceTest.java > % GestionServiceTest > obtenerPedidoCompletoTest()

1:2 import org.junit.jupiter.api.test;

1:3 import org.junit.jupiter.api.extension.ExtendWith;

1:4 import org.mockito.InjectMocks;

1:5 import org.mockito.Mock;

1:6 import org.mockito.Mockito;

1:7 import org.mockito.junit.jupiter.MockitoExtension;

1:8 import org.springframework.web.reactive.function.client.WebClient;

1:9

2:0 import com.donutin.servicio_pedido.model.DetallePedido;

2:1 import com.donutin.servicio_pedido.model.PedidoDTO;

2:2 import com.donutin.servicio_pedido.repository.DetallePedidoRepository;

2:3

2:4 import reactor.core.publisher.Mono;

2:5

2:6 @ExtendWith(MockitoExtension.class)

2:7 @SuppressWarnings("rawtypes") // Para eliminar las advertencias de RequestHeaders

2:8 public class GestionServiceTest

2:9 {

3:0 // Haremos una prueba unitaria con Pedido

3:1 @Mock

3:2 private DetallePedidoRepository detallePedidoRepository;

3:3 @Mock

3:4 private WebClient.Builder webClientBuilder;

3:5 @InjectMocks

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

TEST RESULTS

PORTS

%TESTC 2 v2

%TSTTREE2,com.donutin.servicio_pedido.ServicioPedidoApplicationTests,true,1,false,1,ServicioPedidoApplicationTests,

,[engine:junit-jupiter]/[class:com.donutin.servicio_pedido.ServicioPedidoApplicationTests]

%TSTTREE3,contextLoads(com.donutin.servicio_pedido.ServicioPedidoApplicationTests),false,1,false,2,contextLoads(),,

,[engine:junit-jupiter]/[class:com.donutin.servicio_pedido.ServicioPedidoApplicationTests]/[method:contextLoads()]

%TSTTREE4,com.donutin.servicio_pedido.service.GestionServiceTest,true,1,false,1,GestionServiceTest,,[engine:junit-j

upiter]/[class:com.donutin.servicio_pedido.service.GestionServiceTest]

%TSTTREE5,obtenerPedidoCompletoTest(com.donutin.servicio_pedido.service.GestionServiceTest),false,1,false,4,obtener

PedidoCompletoTest(),,[engine:junit-jupiter]/[class:com.donutin.servicio_pedido.service.GestionServiceTest]/[method

:obtenerPedidoCompletoTest()]

%TESTS 3,contextLoads(com.donutin.servicio_pedido.ServicioPedidoApplicationTests)

%TESTE 3,contextLoads(com.donutin.servicio_pedido.ServicioPedidoApplicationTests)

%TESTS 5,obtenerPedidoCompletoTest(com.donutin.servicio_pedido.service.GestionServiceTest)

%TESTE 5,obtenerPedidoCompletoTest(com.donutin.servicio_pedido.service.GestionServiceTest)

%RUNTIME14880

Test Runner for Java

✓ ServicioPedidoApplicationTests \$(symbol-namespace) com.donutin...

✓ GestionServiceTest \$(symbol-namespace) com.donutin.servicio_pedi...

✓ contextLoads() \$(symbol-class) ServicioPedidoApplicationTests + \$(sy...

✓ obtenerPedidoCompletoTest() \$(symbol-class) GestionServiceTest + ...

Durante el proceso de despliegue y testing en esta fase, se abordaron y depuraron fallos críticos de integración:

Filtros de Seguridad Interrumpidos: Se corrigieron las rutas de importación incorrectas presentes en las configuraciones de seguridad locales de los microservicios.

Simulación Reactiva con WebClient: En la capa de pruebas (*Test.java), específicamente en el módulo de Logística, Pagos y Promociones, se definió la configuración de mocks utilizando Mockito. Se resolvieron las advertencias de seguridad de tipos al simular el comportamiento reactivo de WebClient. Para ello, se corrigió el encadenamiento de respuestas simuladas al consumir flujos síncronos y asíncronos distribuidos en llamadas cruzadas.

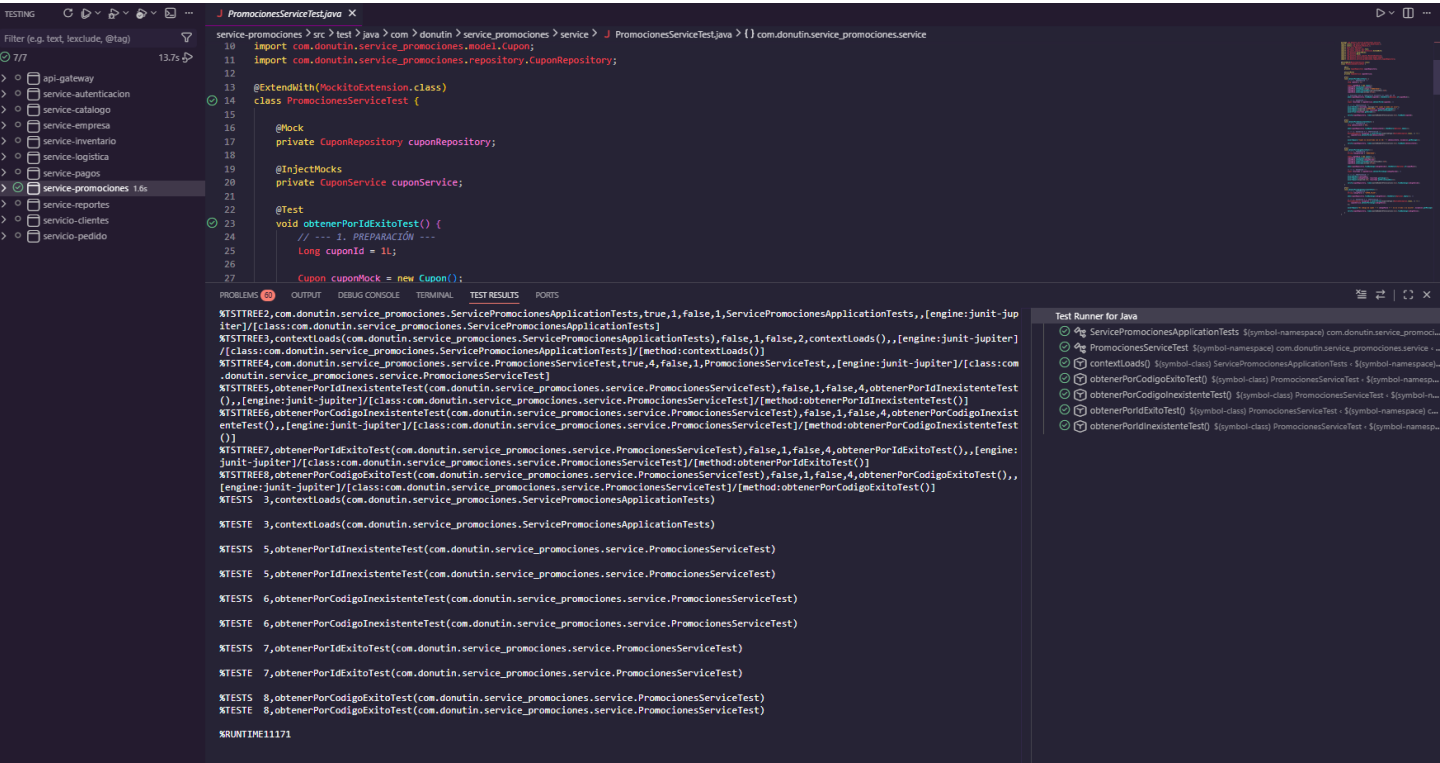
service-promociones

En esta prueba unitaria simulamos que cuando el repositorio CuponRepository guarde cualquier cupón de descuento, retorne el mismo objeto incluyendo su ID autoincremental generado, para evitar el uso y la dependencia de una base de datos real. En la preparación del entorno de prueba se inicializan atributos críticos como el código alfanumérico único, el porcentaje de descuento y su estado lógico activo.

```
service-promociones > src > test > java > com > donutin > service_promociones > service > J PromocionesServiceTest.java > {} com.donutin.service_promociones.service
1 package com.donutin.service_promociones.service;
2 import static org.junit.jupiter.api.Assertions.*;
3 import static org.mockito.Mockito.*;
4 import java.util.Optional;
5 import org.junit.jupiter.api.Test;
6 import org.junit.jupiter.api.extension.ExtendWith;
7 import org.mockito.InjectMocks;
8 import org.mockito.Mock;
9 import org.mockito.junit.jupiter.MockitoExtension;
10 import com.donutin.service_promociones.model.Cupon;
11 import com.donutin.service_promociones.repository.CuponRepository;
12
13 @ExtendWith(MockitoExtension.class)
14 class PromocionesServiceTest {
15
16     @Mock
17     private CuponRepository cuponRepository;
18
19     @InjectMocks
20     private CuponService cuponService;
21
22     @Test
23     void obtenerPorIdExitoTest() {
24         // --- 1. PREPARACIÓN ---
25         Long cuponId = 1L;
26
27         Cupon cuponMock = new Cupon();
28         cuponMock.setIdCupon(cuponId);
29         cuponMock.setCodigo(codigo: "DONUT2026");
30         cuponMock.setPorcentajeDesc(porcentajeDesc: 15);
31         cuponMock.setEstado(estado: true);
32
33         // Simulamos que el repositorio encuentra el cupón por ID
34         when(cuponRepository.findById(cuponId)).thenReturn(Optional.of(cuponMock));
35
36         // --- 2. EJECUCIÓN ---
37         Cupon resultado = cuponService.obtenerPorId(cuponId); //
38
39         // --- 3. VERIFICACIÓN ---
40         assertNotNull(resultado, message: "El cupón no debe ser nulo");
41         assertEquals(expected: "DONUT2026", resultado.getCodigo()); //
42         assertEquals(expected: 15, resultado.getPorcentajeDesc()); //
43         assertTrue(resultado.getEstado());
44
45         verify(cuponRepository, times(wantedNumberOfInvocations: 1)).findById(cuponId);
46     }
47
48     @Test
49     void obtenerPorIdInexistenteTest() {
50         // --- 1. PREPARACIÓN ---
51         Long idInexistente = 99L;
52
53         when(cuponRepository.findById(idInexistente)).thenReturn(Optional.empty());
54
55         // --- 2. EJECUCIÓN Y 3. VERIFICACIÓN ---
56         RuntimeException exception = assertThrows(expectedType: RuntimeException.class, () -> {
57             cuponService.obtenerPorId(idInexistente);
58         });
59
60         assertEquals("Cupón no encontrado con el ID: " + idInexistente, exception.getMessage());
61
62         verify(cuponRepository, times(wantedNumberOfInvocations: 1)).findById(idInexistente);
63     }
64 }
```

```
64
65 @Test
66 void obtenerPorCodigoExitoTest() {
67     // --- 1. PREPARACIÓN ---
68     String codigoValido = "DONUT2026";
69
70     Cupon cuponMock = new Cupon(); //
71     cuponMock.setIdCupon(idCupon: 1L);
72     cuponMock.setCodigo(codigoValido);
73     cuponMock.setPorcentajeDesc(porcentajeDesc: 15);
74     cuponMock.setEstado(estado: true);
75
76     when(cuponRepository.findByCodigo(codigoValido)).thenReturn(Optional.of(cuponMock));
77
78     // --- 2. EJECUCIÓN ---
79     Cupon resultado = cuponService.obtenerPorCodigo(codigoValido); //
80
81     // --- 3. VERIFICACIÓN ---
82     assertNotNull(resultado);
83     assertEquals(codigoValido, resultado.getCodigo());
84     assertEquals(expected: 15, resultado.getPorcentajeDesc());
85
86     verify(cuponRepository, times(wantedNumberOfInvocations: 1)).findByCodigo(codigoValido);
87 }
88
89 @Test
90 void obtenerPorCodigoInexistenteTest() {
91     // --- 1. PREPARACIÓN ---
92     String codigoFalso = "CUPON_FALSO";
93
94     when(cuponRepository.findByCodigo(codigoFalso)).thenReturn(Optional.empty()); //
95
96     // --- 2. EJECUCIÓN Y 3. VERIFICACIÓN ---
97     RuntimeException exception = assertThrows(expectedType: RuntimeException.class, () -> {
98         cuponService.obtenerPorCodigo(codigoFalso);
99     });
100
101     assertEquals("El código de cupón " + codigoFalso + " no es válido o no existe", exception.getMessage());
102
103     verify(cuponRepository, times(wantedNumberOfInvocations: 1)).findByCodigo(codigoFalso);
104 }
105 }
```


Resultado del testeo:



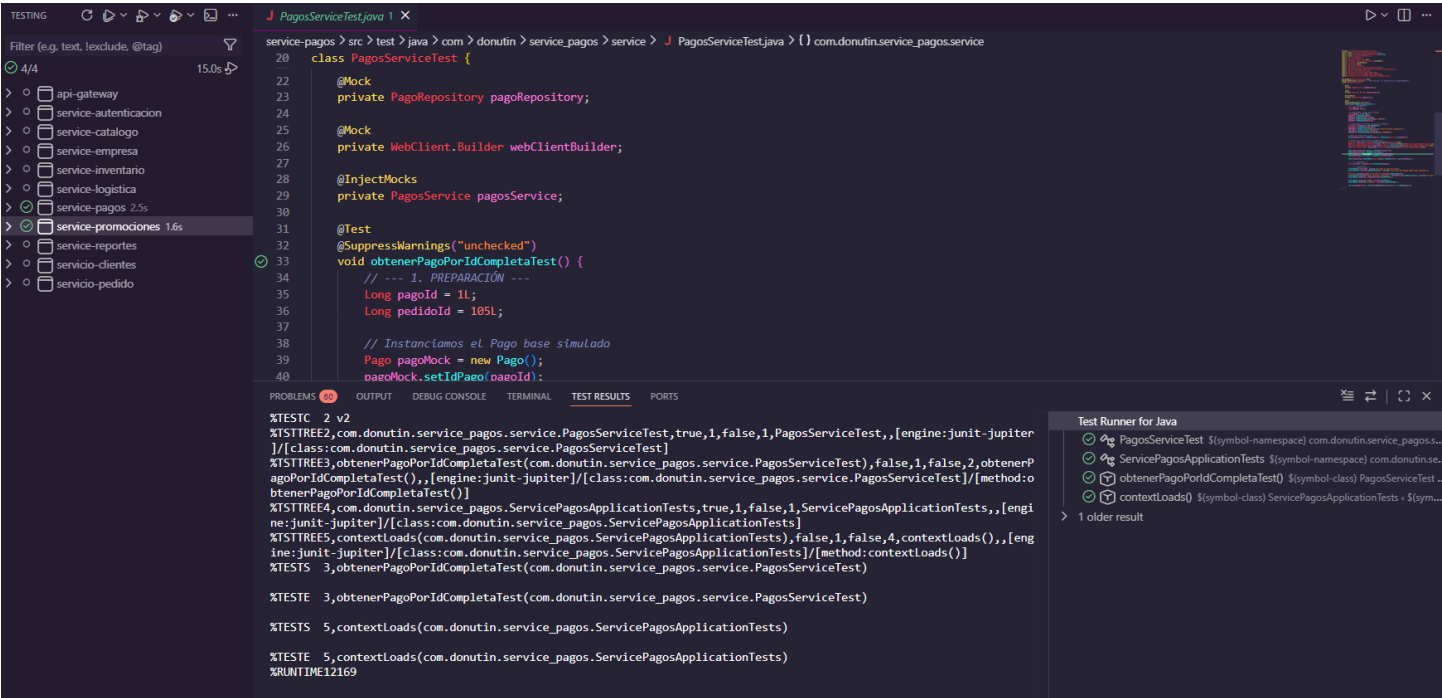
service-pagos (con WebClient)

Testear el PagoService de service-pagos representó un desafío avanzado debido a la implementación de WebClient para el consumo de datos síncronos y asíncronos cruzados en tiempo real. Para evitar realizar peticiones HTTP reales hacia el microservicio productor (service-pedido), se recurrió a un encadenamiento estructurado de mocks utilizando **Mockito**.

Durante el proceso, se corrigieron las advertencias de seguridad de tipos configurando de manera estricta el método .bodyToMono(Object.class) para que resolviera y retornara una estructura limpia y aislada basada en los objetos simulados pedidoMock y clienteMock. De esta manera, se garantiza la estabilidad del test unitario ante flujos reactivos. En esta prueba, simularemos la respuesta.



Resultado del testeo:



service-logistica (con WebClient)

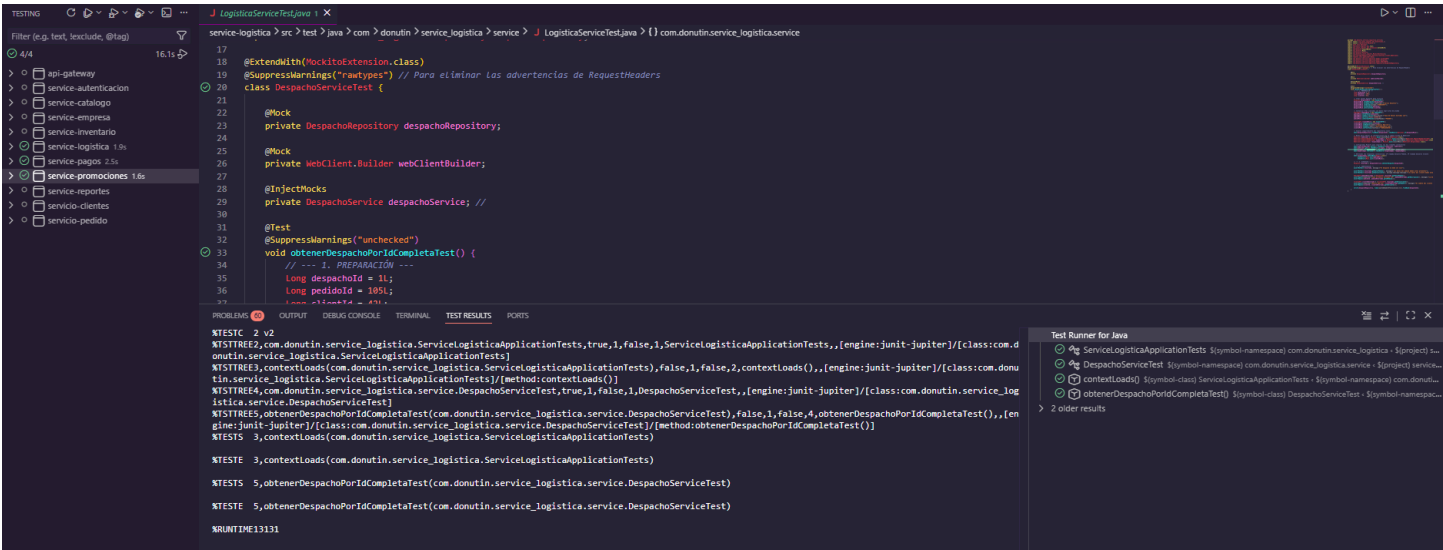
Testear el DespachoService de service-logistica al igual que PagoService, también representó un desafío avanzado debido a la implementación de WebClient para el consumo de datos síncronos y asíncronos cruzados en tiempo real. Para evitar realizar peticiones HTTP reales hacia el microservicio productor (service-pedido), se recurrió a un encadenamiento estructurado de mocks utilizando **Mockito**.

Durante el proceso, se corrigieron las advertencias de seguridad de tipos configurando de manera estricta el método .bodyToMono(Object.class) para que resolviera y retornara una estructura limpia y aislada basada en los objetos simulados pedidoMock y clienteMock. De esta manera, se garantiza la estabilidad del test unitario ante flujos reactivos.




```
79
80 // --- 2. EJECUCIÓN ---
81 Despacho resultado = despachoService.obtenerDespacho(despachoId);
82
83 // --- 3. VERIFICACIÓN ---
84 assertNotNull(resultado, message: "El despacho no debe ser nulo");
85
86 assertNotNull(resultado.getDatosPedido(), message: "Los datos del pedido deben estar presentes");
87 assertNotNull(resultado.getDatosCliente(), message: "Los datos del cliente deben estar presentes");
88
89 PedidoDTO pedidoRetornado = (PedidoDTO) resultado.getDatosPedido();
90 assertEquals(expected: "Caja de Donuts Surtidas x12", pedidoRetornado.getDescripcion(), message: "La descripción del pedido debe coincidir");
91 assertEquals(pedidoId, pedidoRetornado.getIdPedido());
92
93 ClienteDTO clienteRetornado = (ClienteDTO) resultado.getDatosCliente();
94 assertEquals(expected: "Danilo Ramirez", clienteRetornado.getNombre(), message: "El nombre del cliente debe coincidir");
95 assertEquals(clienteId, clienteRetornado.getIdCliente());
96
97 verify(despachoRepository, times(wantedNumberOfInvocations: 1)).findById(despachoId);
98 }
99
100
```

Resultado del testeo:



CONCLUSIÓN GENERAL DE LA ARQUITECTURA

El estado final del proyecto consolida un ecosistema robusto de 10 microservicios independientes coordinados de manera eficiente a través de una API Gateway centralizada. La incorporación de la validación de tokens JWT en la frontera del sistema, sumada a la autogeneración de documentación interactiva por Swagger, garantiza que la plataforma no solo cumple con altas directrices de seguridad y desacoplamiento, sino que también ofrece un entorno de desarrollo escalable y transparente listo para su integración con aplicaciones frontend.